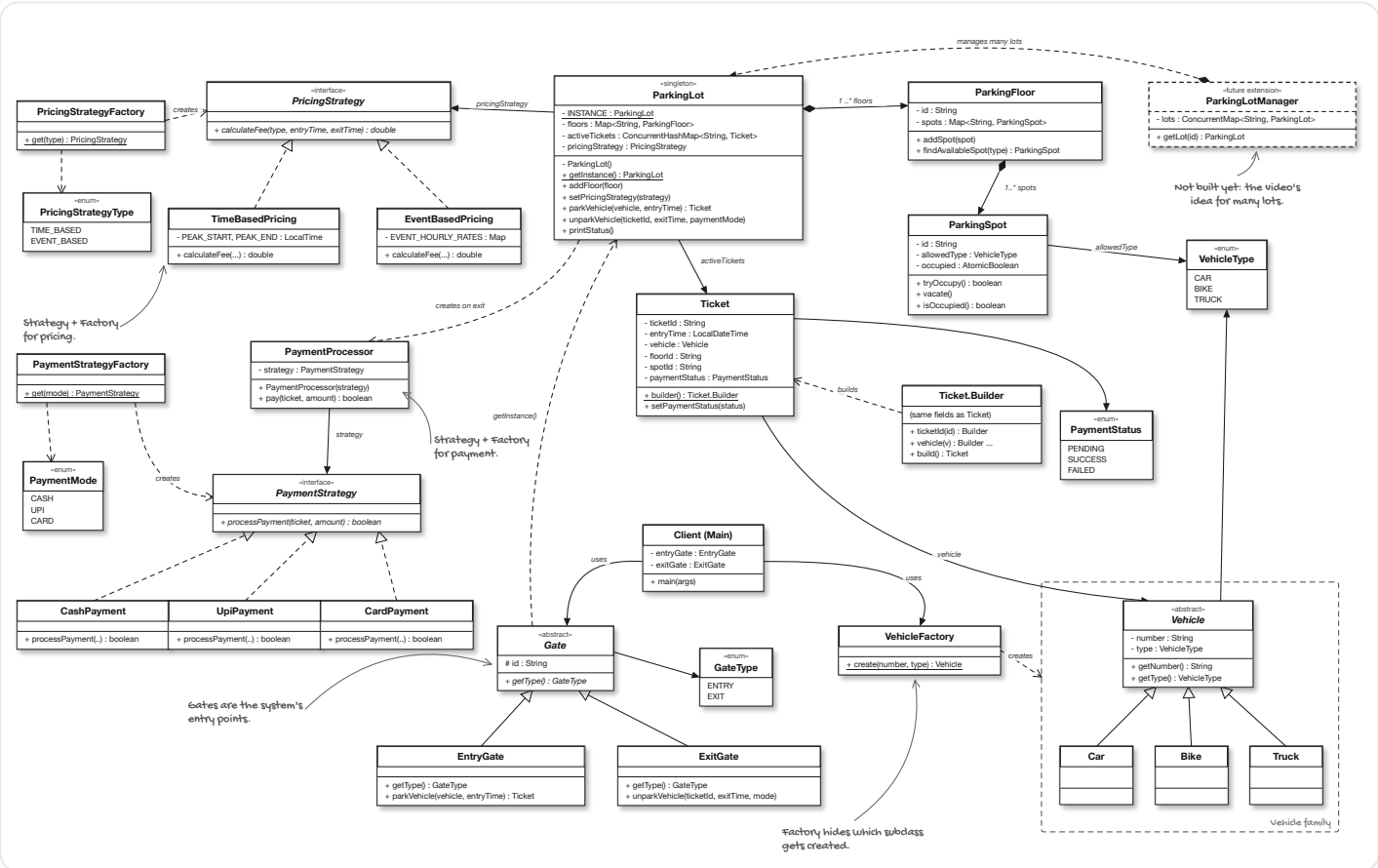
LOW LEVEL DESIGN · CASE STUDY

Parking Lot System Design

Study notes that build the full UML and Java code in six progressive phases: requirements, design patterns, concurrency and a complete walk-through of the code flow.



Based on "Design Parking Lot | LLD, UML, Concurrency & Code Explained" by Shubh Patel (YouTube)
and the reference code at github.com/shubhkpatel/Low-Level-Design (parking_lot package)

1 2 3 4 5 6

How to use these notes

We never draw the finished diagram first. We start with the smallest useful piece (a vehicle and a spot) and add one idea per phase. Each phase has its own folder of **complete, compilable Java code**, so you can open it, run it and compare it with the previous phase.

1 — Vehicles & Parking Spots
Abstraction · Inheritance · Enums
`Phase-1-Vehicles-and-Spots`

2 — Floors, ParkingLot & Ticket
Composition · Singleton · Builder
`Phase-2-Floors-Lot-and-Tickets`

3 — VehicleFactory & Gates
Factory · Delegation · Client
`Phase-3-Factory-and-Gates`

4 — Pricing Strategy
Strategy · Open/Closed
`Phase-4-Pricing-Strategy`

5 — Payment Strategy
Strategy again · Dependency Inversion
`Phase-5-Payment-Strategy`

6 — Concurrency
Race conditions · CAS · ConcurrentHashMap
`Phase-6-Concurrency`

In every phase you get

- **The goal**: which requirement this phase satisfies.
- **A UML diagram** of everything built so far. New classes are tinted and tagged NEW; classes that gained members are tagged UPDATED, with the new lines highlighted.
- **Concepts and patterns**, with code snippets taken from that phase's folder.
- **Real output** from running the phase.
- **A checkpoint**: questions to answer before you move on.

Folder map

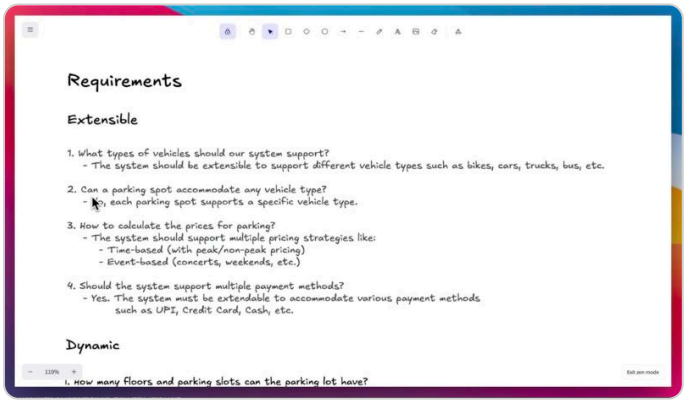
```
ParkingLot-LLD-Study/
├─ ParkingLot_LLD_Study_Notes.pdf   ← this file
├─ Phase-1-Vehicles-and-Spots/
│  ├─ run.sh                        ← compile + run
│  └─ src/parkinglot/...
├─ Phase-2-Floors-Lot-and-Tickets/
├─ Phase-3-Factory-and-Gates/
├─ Phase-4-Pricing-Strategy/
├─ Phase-5-Payment-Strategy/
├─ Phase-6-Concurrency/             ← final design
├─ diagrams/      (SVG of every UML)
└─ screenshots/   (frames from the video)
```

Run any phase with `bash Phase-N-.../run.sh` (JDK 17+). The code is plain Java with no Lombok, so it compiles without Maven or Gradle.

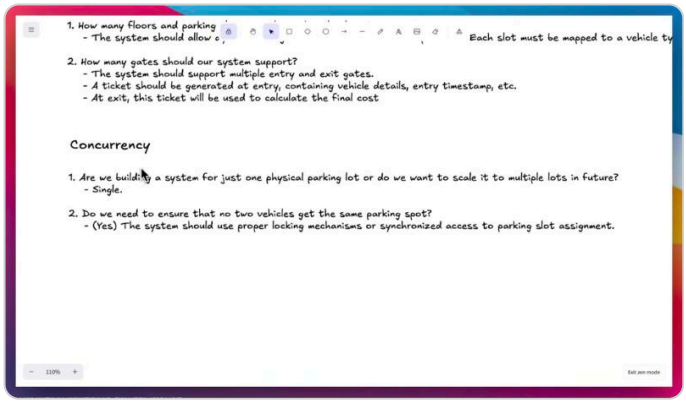
Study loop that works
Read the phase → sketch the UML yourself on paper → open the folder and type the new classes from memory → run `run.sh` → compare. Retyping teaches more than reading.

Step 0: Clarify the requirements

An LLD interview starts with questions, not classes. The video groups them into three buckets: **Extensible** (what will change?), **Dynamic** (what is configured at runtime?) and **Concurrency** (what happens when things run at the same time?).



Video @ 8:42: the "Extensible" questions



Video @ 13:20: the "Dynamic" and "Concurrency" questions

From requirement to design decision

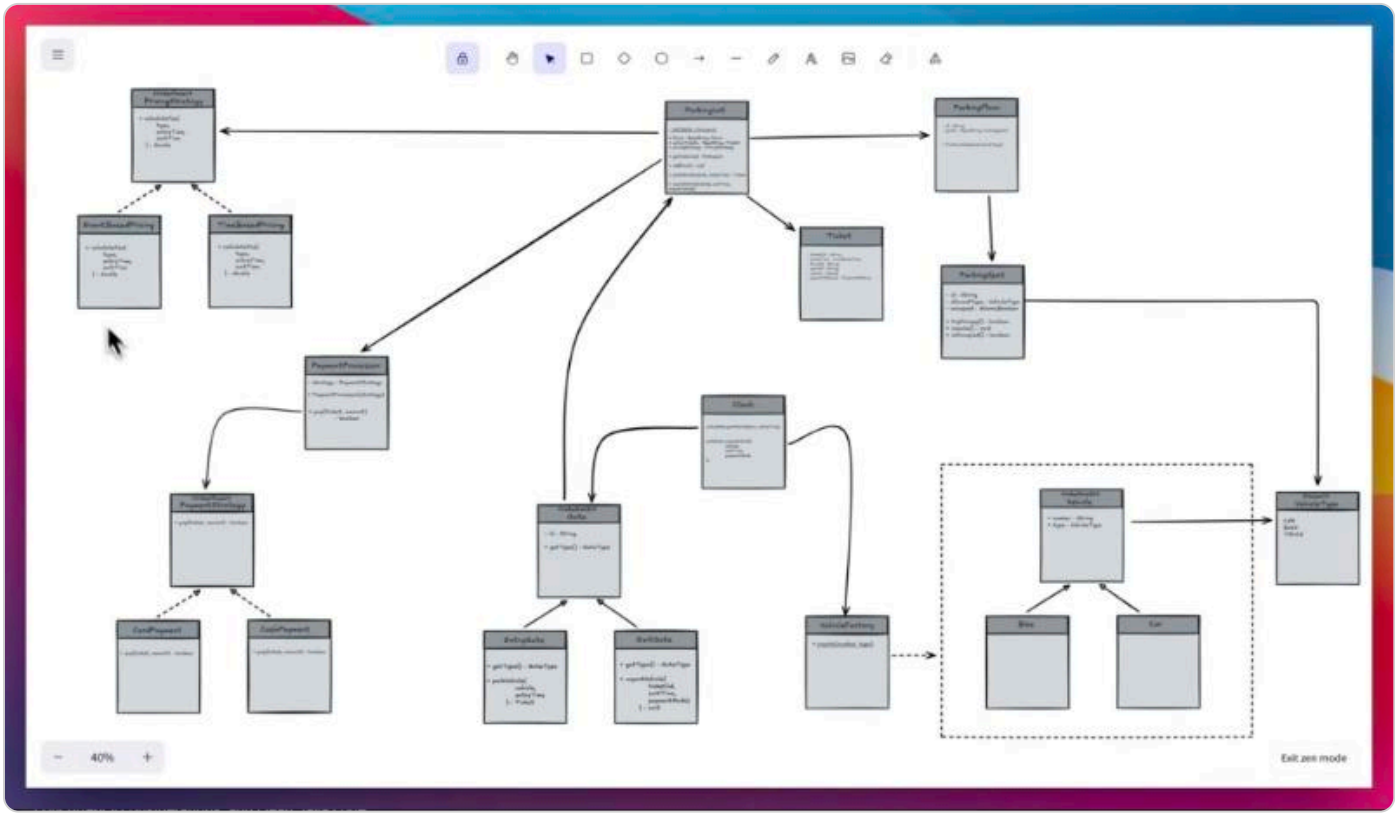
#	Requirement (from the video)	Design decision	Phase
E1	Support bikes, cars, trucks, and later bus etc.	Abstract `Vehicle` + subclasses + `VehicleType` enum; `VehicleFactory`	1 3
E2	Each spot accepts one specific vehicle type	`ParkingSpot.allowedType`	1
E3	Many pricing schemes: time-based (peak/off-peak), event-based	`PricingStrategy` interface (Strategy pattern)	4
E4	Many payment methods: UPI, card, cash...	`PaymentStrategy` interface + `PaymentProcessor`	5
D1	Configurable floors and spots per floor	`ParkingLot` ◆ `ParkingFloor` ◆ `ParkingSpot` (composition) with `addFloor` / `addSpot`	2
D2	Multiple entry/exit gates; ticket issued at entry, used at exit to compute the cost	`Gate` → `EntryGate` / `ExitGate`; `Ticket` (Builder)	2 3
C1	A single physical lot (for now)	`ParkingLot` is a **Singleton**	2
C2	Two vehicles must never get the same spot	`AtomicBoolean.compareAndSet` in `tryOccupy()`; `ConcurrentHashMap` for tickets	6

Interview tip

Say the requirements back as a numbered list (like the table above) before drawing anything. Then every class you add can point at a requirement, which shows the interviewer your design is driven by needs and not habit.

The destination: the UML from the video

This is the whole board from the video (≈19:10). It's small here on purpose: you don't need to read it yet. By Phase 6 you will have rebuilt every box on it, one phase at a time. The redrawn, readable version is on the cover and at the end of these notes.



Video @ 19:10: the complete class diagram drawn in the video

How to read the diagrams in these notes

Inheritance (extends): Car *is a* Vehicle

Realization (implements an interface)

Association (has a reference to / uses)

Composition: the part can't exist without the whole

Dependency (creates / calls, no field kept)

`+ public - private # protected`
underlined = static, *italic* = abstract

Your toolkit: the patterns in this design

Singleton Phase 2

Problem: one physical lot, so there should be one shared `ParkingLot` object for every gate.

Shape: private constructor + `static final INSTANCE` + `getInstance()`. Eager init is thread-safe for free (class loading is synchronized).

Trade-off: global state is hard to test and blocks "many lots" → see *ParkingLotManager*.

Builder Phase 2

Problem: a `Ticket` has many fields; a 5-arg constructor is easy to call in the wrong order.

Shape: `Ticket.builder().ticketId(..).vehicle(..)… build()`. Named steps, validation in `build()`, immutable result.

Original code: Lombok `@Builder` generates this for you.

Simple Factory Phases 3–5

Problem: callers shouldn't `new Car()` / `new Bike()` or know concrete strategy classes.

Shape: one static method maps an enum to an object: `VehicleFactory`, `PricingStrategyFactory`, `PaymentStrategyFactory`.

Win: adding a type touches one `switch`, and the compiler flags a missing case.

Strategy Phases 4–5

Problem: pricing and payment rules vary and must be swappable at runtime.

Shape: an interface (`PricingStrategy`, `PaymentStrategy`) + one class per rule. The context (`ParkingLot`, `PaymentProcessor`) holds the *interface*.

Win: new rule = new class. Existing code is not edited (Open/Closed).

Principles you will see again and again

Single Responsibility	`PaymentProcessor` only pays; pricing math lives in a strategy; gates only route requests.
Open/Closed	New vehicle, pricing rule or payment mode = add a class (plus an enum value), don't edit `ParkingLot`.
Liskov Substitution	Any `Vehicle` subclass or any strategy can be passed wherever the parent type is expected.
Dependency Inversion	`ParkingLot` depends on `PricingStrategy` (abstraction), not on `TimeBasedPricing`.
Composition over inheritance	The lot *has* floors, which *have* spots. Behaviour is plugged in (strategies), not inherited.

PHASE 1

Vehicles & Parking Spots

Abstraction · Inheritance · Enums

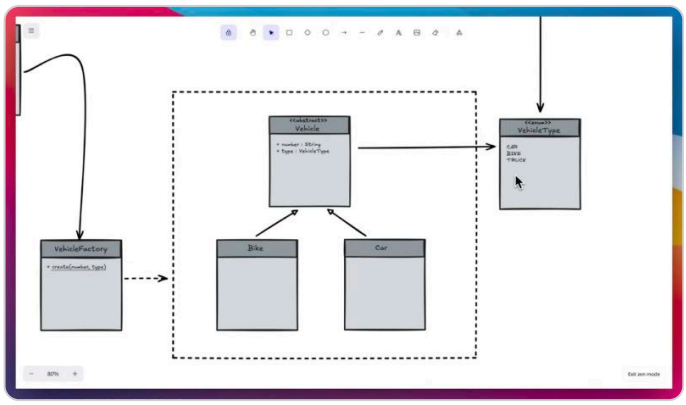
Model the two nouns everything else depends on: the vehicle that arrives and the spot it parks in.

`Phase-1-Vehicles-and-Spots/`

`+ VehicleType (enum)` `+ Vehicle (abstract)` `+ Car` `+ Bike` `+ Truck` `+ ParkingSpot`

Goal

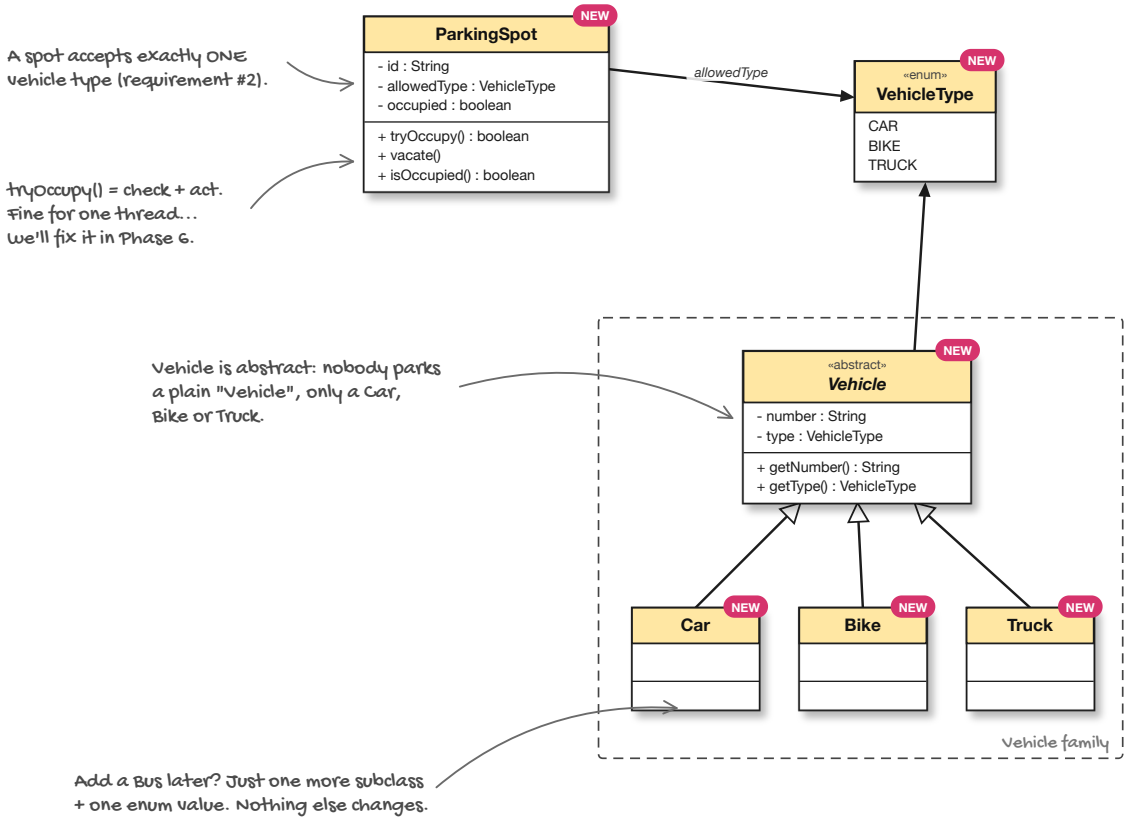
Requirements **E1** (many vehicle types) and **E2** (a spot accepts one type). No lot, no tickets yet: just "can this vehicle use this spot?"



Video @ 23:20: Vehicle hierarchy and the VehicleType enum

Concepts

- **Abstraction:** `Vehicle` is `abstract`. You can't park "a vehicle", only a concrete one.
- **Inheritance:** `Car`, `Bike`, `Truck` only pass their type up to the parent constructor.
- **Enum as a type tag:** `VehicleType` is what spots compare against, so matching is a cheap `==`.
- **Encapsulation:** `occupied` is private and changes only through `tryOccupy()` / `vacate()`.



model/Vehicle.java

```
public abstract class Vehicle {
    private final String number;
    private final VehicleType type;

    protected Vehicle(String number, VehicleType
type) {
        this.number = number;
        this.type = type;
    }

    public String getNumber() { return number; }
    public VehicleType getType() { return type; }

    @Override
    public String toString() { return type + "(" +
number + ")"; }
}
```

model/Car.java (Bike and Truck are identical)

```
public class Car extends Vehicle {
    public Car(String number) { super(number,
VehicleType.CAR); }
}
```

enums/VehicleType.java

```
public enum VehicleType {
    CAR, BIKE, TRUCK
}
```

model/ParkingSpot.java

```
public class ParkingSpot {
    private final String id;
    private final VehicleType allowedType;
    private boolean occupied = false;

    public ParkingSpot(String id, VehicleType
allowedType) {
        this.id = id;
        this.allowedType = allowedType;
    }

    // Check-then-act: correct for one thread, NOT
thread-safe (fixed in Phase 6).
    public boolean tryOccupy() {
        if (occupied) return false;
        occupied = true;
        return true;
    }

    public void vacate() { occupied = false; }

    public boolean isOccupied() { return occupied; }
    public String getId() { return id; }
    public VehicleType getAllowedType() { return
allowedType; }
}
```

Why `tryOccupy()` and not `setOccupied(true)` ? It **checks and claims** in one call and tells you whether you got the spot. That shape is what lets Phase 6 make it thread-safe without changing any caller.

```
▶ bash Phase-1-Vehicles-and-Spots/run.sh

═══ Phase 1: Vehicles & Parking Spots ═══
  CAR(KA01AB1234) → S2
  BIKE(KA01XY0001) → S1
  CAR(KA05ZZ9999) → NO SPOT
  Spot S1 [BIKE] Occupied
  Spot S2 [CAR] Occupied
  Spot S3 [TRUCK] Free
```

Why an enum AND subclasses?

The subclass gives each vehicle its own type (room for type-specific behaviour later). The enum is a lightweight key that spots, pricing tables and factories can switch on without `instanceof` checks.

Checkpoint

1) What breaks if `Vehicle` were not abstract? 2) Add a `Bus` : which files change? (Answer: `Bus.java` + one enum value. Later phases add one line to each factory/pricing `switch` , and the compiler points them out.)

PHASE 2

Floors, ParkingLot & Ticket

Composition · Singleton · Builder

Give the spots a home. Build the structure lot → floors → spots, make the lot a Singleton and hand out a Ticket on entry.

`Phase-2-Floors-Lot-and-Tickets/`

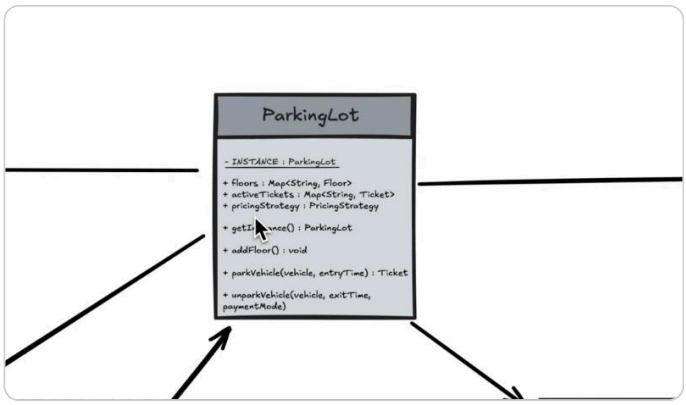
`+ ParkingFloor` `+ ParkingLot «singleton»` `+ Ticket + Ticket.Builder` `+ DateTimeParser` `~ Main`

Goal

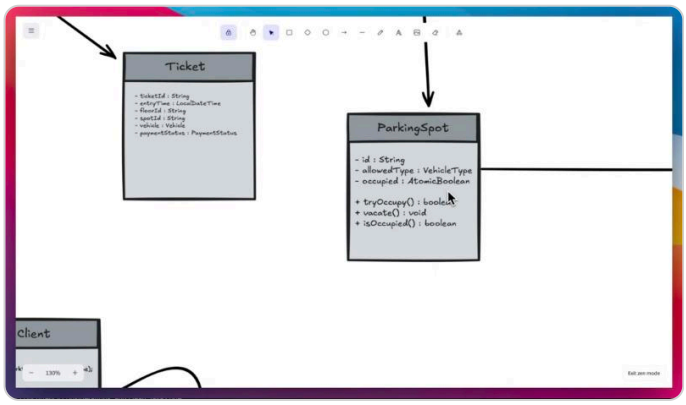
Requirements **D1** (configurable floors/spots), **D2** (ticket at entry) and **C1** (one lot).

Concepts

- **Composition** (filled diamond): a `ParkingFloor` belongs to one lot, a `ParkingSpot` to one floor. Maps keyed by id give O(1) lookup at exit.
- **Singleton:** every gate calls `ParkingLot.getInstance()` and sees the same floors and tickets.
- **Builder:** the ticket is assembled step by step and checked in `build()`.
- **Ticket as the link:** it stores `floorId` + `spotId`, so the exit finds the exact spot to free without searching.

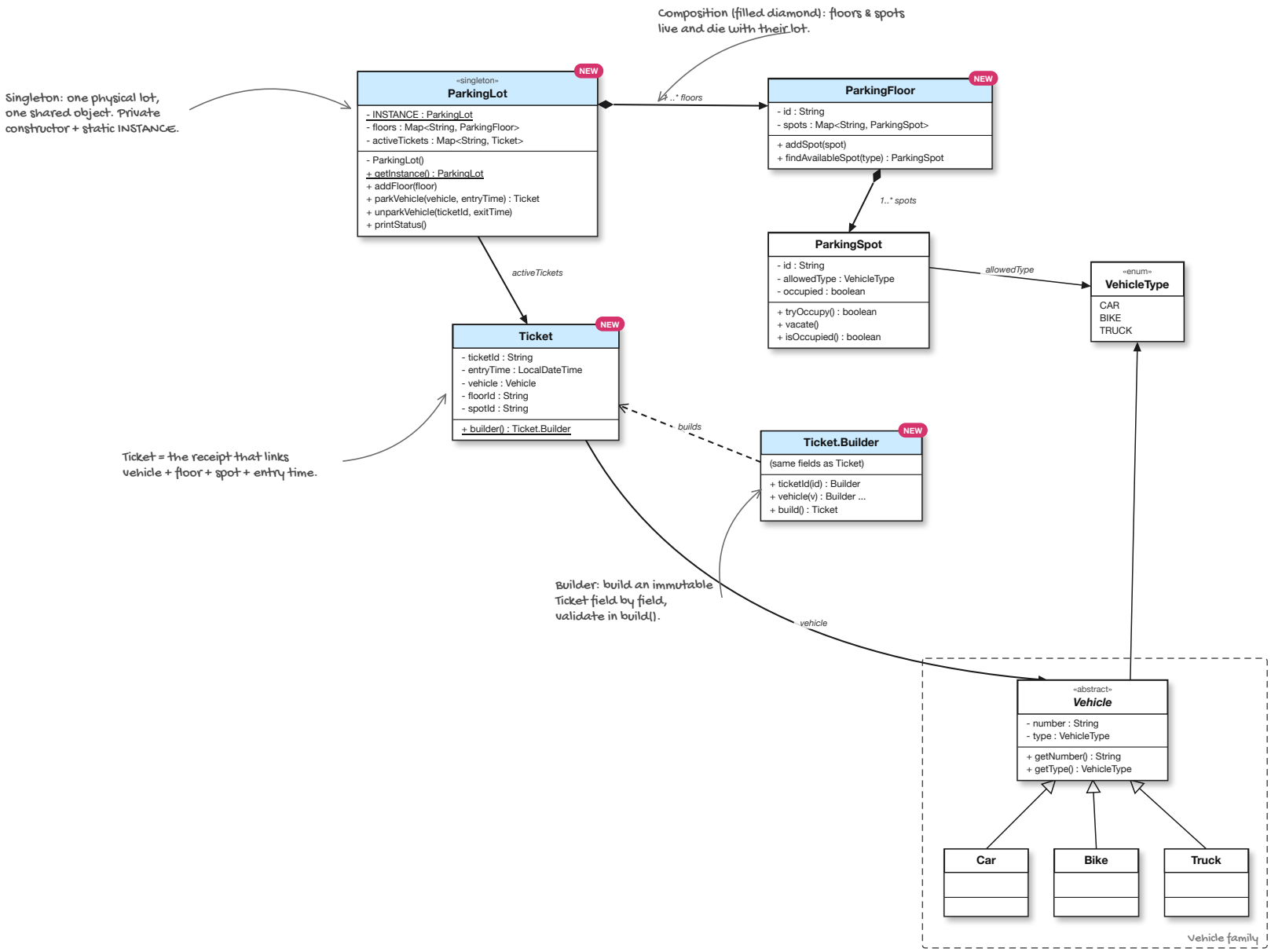


Video @ 27:20: ParkingLot fields and methods



Video @ 26:00: Ticket and ParkingSpot

Phase 2 UML: the structure appears



service/ParkingLot.java: the Singleton part

```java
public class ParkingLot {
    private static final ParkingLot INSTANCE = new
ParkingLot();

    private final Map<String, ParkingFloor> floors =
new LinkedHashMap<>();
    private final Map<String, Ticket> activeTickets
= new LinkedHashMap<>();

    private ParkingLot() {}

    public static ParkingLot getInstance() {
        return INSTANCE;
    }
    // ...
}
```

Eager initialisation: the JVM creates `INSTANCE` once when the class loads, and class loading is thread-safe. No `synchronized`, no double-checked locking needed.

model/ParkingFloor.java

```java
public ParkingSpot findAvailableSpot(VehicleType
vehicleType) {
    for (ParkingSpot spot : spots.values()) {
        if (spot.getAllowedType() == vehicleType &&
spot.tryOccupy()) {
            return spot;
        }
    }
    return null;
}
```

service/ParkingLot.java: parkVehicle

```java
public Ticket parkVehicle(Vehicle vehicle,
LocalDateTime entryTime) {
    for (ParkingFloor floor : floors.values()) {
        ParkingSpot spot =
floor.findAvailableSpot(vehicle.getType());
        if (spot != null) {
            String ticketId = "T-" +
UUID.randomUUID().toString().substring(0, 8);
            Ticket ticket = Ticket.builder()
                    .ticketId(ticketId)
                    .entryTime(entryTime)
                    .vehicle(vehicle)
                    .floorId(floor.getId())
                    .spotId(spot.getId())
                    .build();
            activeTickets.put(ticketId, ticket);
            System.out.println("  " + vehicle + "
parked at " + floor.getId() + "/" + spot.getId()
                    + ". Ticket: " + ticketId);
            return ticket;
        }
    }
    System.out.println("  No spot available for
vehicle type: " + vehicle.getType());
    return null;
}
```

Builder in use

```
Ticket ticket = Ticket.builder()
        .ticketId(ticketId)
        .entryTime(entryTime)
        .vehicle(vehicle)
        .floorId(floor.getId())
        .spotId(spot.getId())
        .build();    // validates required fields
```

model/Ticket.java: Builder.build()

```
public Ticket build() {
    if (ticketId == null || vehicle == null ||
entryTime == null) {
        throw new IllegalStateException("ticketId,
vehicle and entryTime are required");
    }
    return new Ticket(this);
}
```

▶ bash Phase-2-Floors-Lot-and-Tickets/run.sh

```
=== Phase 2: Floors, ParkingLot (Singleton) & Ticket (Builder) ===
  Same instance everywhere? true
  CAR(KA01AB1234) parked at Floor1/F1S2. Ticket: T-2dd8f43a
  CAR(KA02CD5678) parked at Floor2/F2S1. Ticket: T-572f03c1
  No spot available for vehicle type: CAR
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Occupied
  Floor: Floor2
    Spot F2S1 [CAR] - Occupied
  CAR(KA01AB1234) exited after 345 min.
  Invalid ticket ID.
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Free
  Floor: Floor2
    Spot F2S1 [CAR] - Occupied
```

Original code uses HashMap, we use LinkedHashMap

`HashMap` iteration order is not defined, so “first free spot” could come from any floor. `LinkedHashMap` keeps insertion order, so Floor1 fills before Floor2 and the demo output is predictable.

Checkpoint

1) Why does `findAvailableSpot` reserve the spot instead of just returning a free one? 2) What does `parkVehicle` return when the lot is full? 3) Name one downside of the Singleton (hint: unit tests, multiple lots).

PHASE 3

VehicleFactory & Gates

Factory · Delegation · Client

Put doors on the building. Clients enter and leave through Gates, and a Factory turns (number, type) into the right Vehicle.

`Phase-3-Factory-and-Gates/`

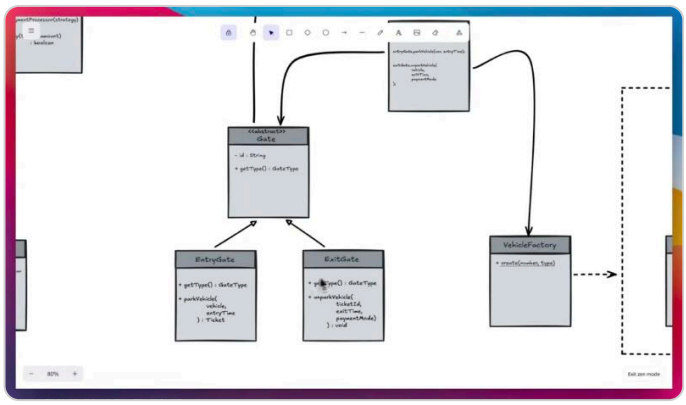
`+ Gate (abstract)` `+ EntryGate` `+ ExitGate` `+ GateType (enum)` `+ VehicleFactory` `+ Client (Main)` `~ Main`

Goal

Requirement **D2**: multiple entry and exit gates. Requirement **E1** again: creating vehicles should not spread `new Car(..)` everywhere.

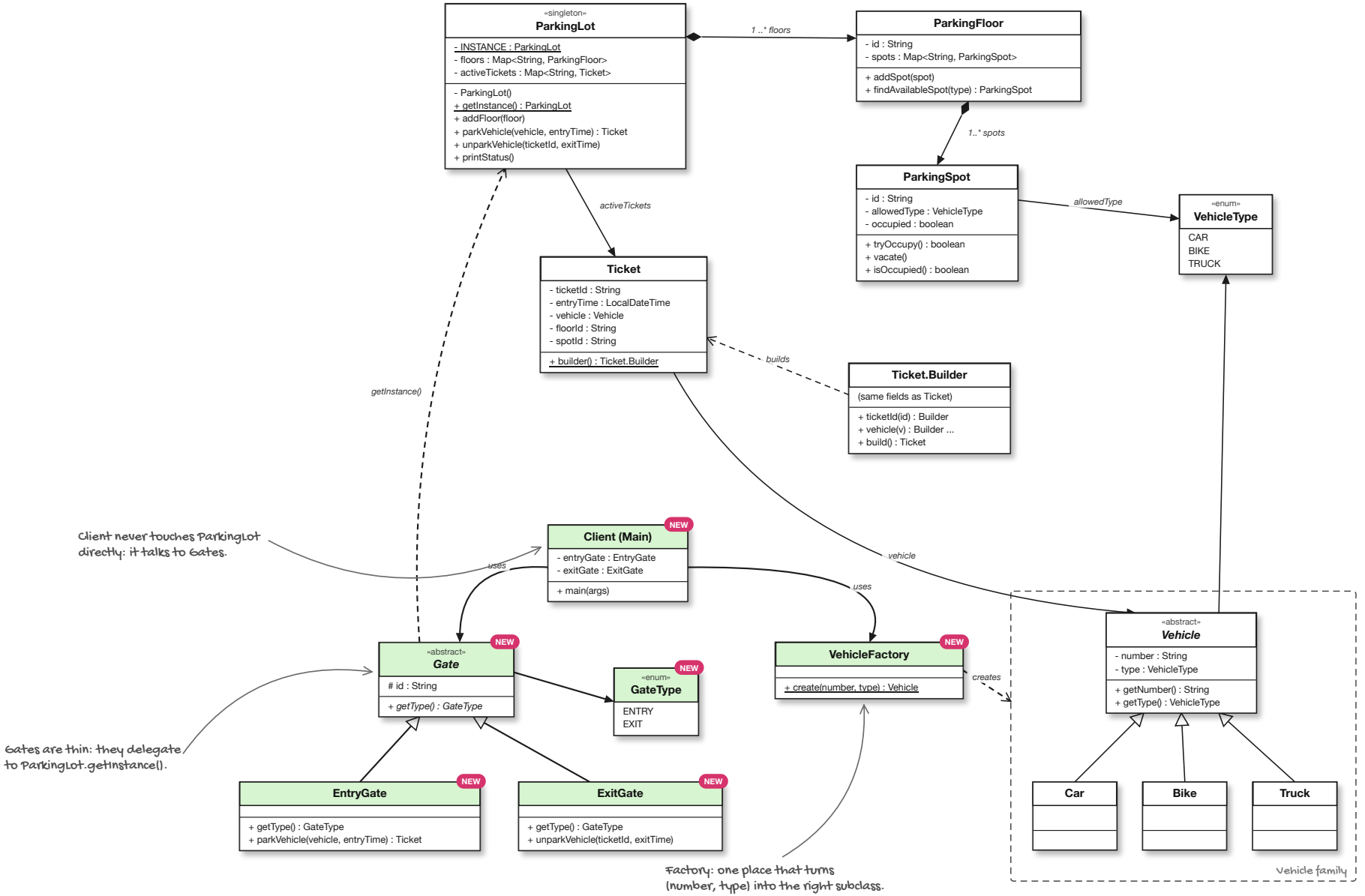
Concepts

- **Abstract class with abstract method:** every `Gate` has an `id` and must say its `GateType`.
- **Delegation:** gates are thin. They forward to `ParkingLot.getInstance()`, so the business rules live in one place.
- **Simple Factory:** `VehicleFactory.create(number, type)` hides which subclass is built. Java's switch expression must cover every enum value, so a new type without a case won't compile.
- **Client:** `Main` plays the user: it only knows gates and the factory.



Video @ 21:30: Gate hierarchy, Client and VehicleFactory

Phase 3 UML: gates and the factory



model/Gate.java

```java
public abstract class Gate {
    protected final String id;

    protected Gate(String id) { this.id = id; }

    public String getId() { return id; }

    public abstract GateType getType();
}
```

factory/VehicleFactory.java

```java
public class VehicleFactory {
    public static Vehicle create(String number,
VehicleType type) {
        return switch (type) {
            case CAR -> new Car(number);
            case BIKE -> new Bike(number);
            case TRUCK -> new Truck(number);
        };
    }
}
```

model/EntryGate.java

```java
public class EntryGate extends Gate {

    public EntryGate(String id) { super(id); }

    @Override
    public GateType getType() { return
GateType.ENTRY; }

    public Ticket parkVehicle(Vehicle vehicle,
LocalDateTime entryTime) {
        return
ParkingLot.getInstance().parkVehicle(vehicle,
entryTime);
    }
}
```

model/ExitGate.java (Phase 3 version)

```java
public class ExitGate extends Gate {

    public ExitGate(String id) { super(id); }

    @Override
    public GateType getType() { return
GateType.EXIT; }

    public void unparkVehicle(String ticketId,
LocalDateTime exitTime) {

ParkingLot.getInstance().unparkVehicle(ticketId,
exitTime);
    }
}
```

```
▶ bash Phase-3-Factory-and-Gates/run.sh

═══ Phase 3: VehicleFactory & Entry/Exit Gates ═══
  CAR(KA01AB1234) parked at Floor1/F1S2. Ticket: T-3c6afd86
  TRUCK(KA09TR0001) parked at Floor1/F1S3. Ticket: T-099f64f7
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Occupied
    Spot F1S3 [TRUCK] - Occupied
  CAR(KA01AB1234) exited after 345 min.
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Free
    Spot F1S3 [TRUCK] - Occupied
```

Checkpoint

1) Why do gates call `getInstance()` instead of holding a `ParkingLot` field? (Both work. A field injected through the constructor is easier to test.) 2) Where would a “gate display board” or a barrier-arm controller fit?

PHASE 4

Pricing Strategy

Strategy · Open/Closed

Charge money. The rule for how much changes (weekday vs. concert night), so pricing becomes a pluggable Strategy.

`Phase-4-Pricing-Strategy/`

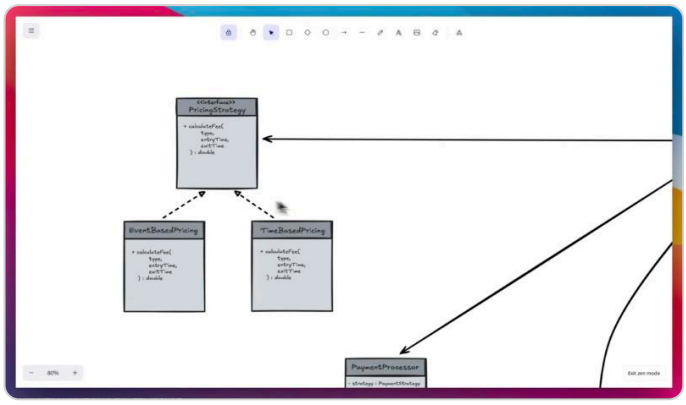
`+ PricingStrategy «interface»` `+ TimeBasedPricing` `+ EventBasedPricing` `+ PricingStrategyFactory`
`+ PricingStrategyType (enum)` `~ ParkingLot (+pricingStrategy, setPricingStrategy)` `~ Main`

Goal

Requirement **E3**: support time-based (peak/off-peak) and event-based pricing, and more later.

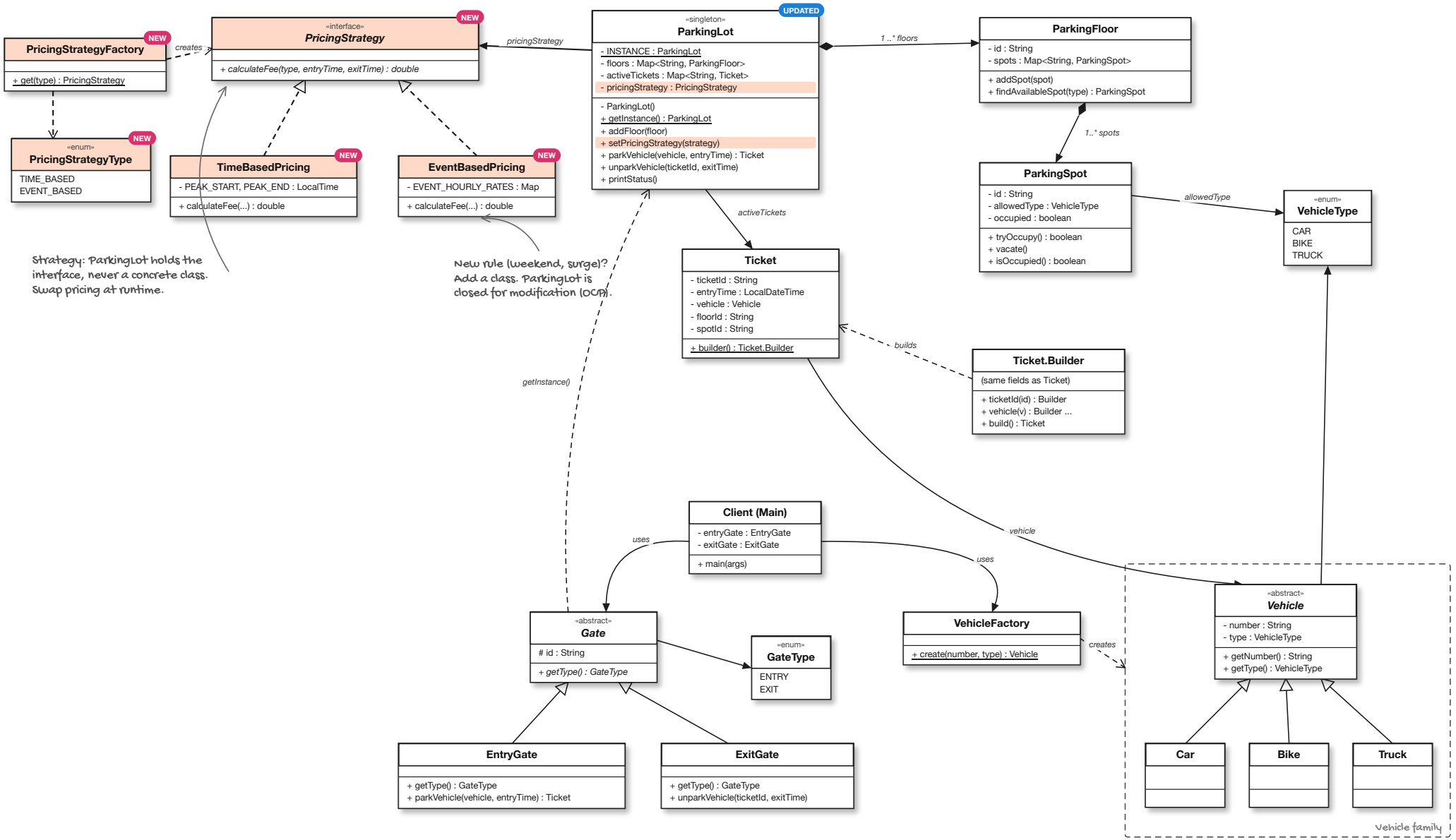
The Strategy pattern, step by step

1. Find what varies: *how the fee is computed*.
2. Put it behind an interface: `calculateFee(type, entryTime, exitTime)`.
3. One class per rule: `TimeBasedPricing`, `EventBasedPricing`.
4. The context (`ParkingLot`) keeps a `PricingStrategy` field and calls it on exit.
5. Swap at runtime with `setPricingStrategy(...)`, e.g. switch to event pricing on match day.



Video @ 19:55: PricingStrategy and its two implementations

Phase 4 UML: pricing plugs into ParkingLot



strategy/pricing/PricingStrategy.java

```java
public interface PricingStrategy {
    double calculateFee(VehicleType type,
LocalDateTime entryTime, LocalDateTime exitTime);
}
```

strategy/pricing/EventBasedPricing.java

```java
public class EventBasedPricing implements
PricingStrategy {
    private static final Map<VehicleType, Double>
EVENT_HOURLY_RATES = Map.of(
            VehicleType.CAR, 50.0,
            VehicleType.BIKE, 30.0,
            VehicleType.TRUCK, 70.0
    );

    @Override
    public double calculateFee(VehicleType type,
LocalDateTime entryTime, LocalDateTime exitTime) {
        long durationMinutes =
Duration.between(entryTime, exitTime).toMinutes();
        long hours = (long)
Math.ceil(durationMinutes / 60.0); // round up to
the next hour
        return EVENT_HOURLY_RATES.getOrDefault(type,
0.0) * hours;
    }
}
```

factory/PricingStrategyFactory.java

```java
public class PricingStrategyFactory {
    public static PricingStrategy
get(PricingStrategyType type) {
        return switch (type) {
            case TIME_BASED -> new
TimeBasedPricing();
            case EVENT_BASED -> new
EventBasedPricing();
        };
    }
}
```

strategy/pricing/TimeBasedPricing.java

```java
public class TimeBasedPricing implements
PricingStrategy {
    private static final LocalTime PEAK_START =
LocalTime.of(8, 0);
    private static final LocalTime PEAK_END =
LocalTime.of(17, 0);

    private boolean isPeak(LocalTime time) {
        return !time.isBefore(PEAK_START) &&
!time.isAfter(PEAK_END);
    }

    @Override
    public double calculateFee(VehicleType type,
LocalDateTime entryTime, LocalDateTime exitTime) {
        if (exitTime.isBefore(entryTime)) throw new
IllegalArgumentException("Exit time before entry
time");

        long durationMinutes =
Duration.between(entryTime, exitTime).toMinutes();
        long totalHours = (long)
Math.ceil(durationMinutes / 60.0);

        int peakHours = 0;
        int nonPeakHours = 0;
        LocalDateTime cursor =
entryTime.truncatedTo(ChronoUnit.HOURS);
        for (int i = 0; i < totalHours; i++) {
            if (isPeak(cursor.toLocalTime()))
peakHours++;
            else nonPeakHours++;
            cursor = cursor.plusHours(1);
        }

        double peakRate = switch (type) {
            case CAR -> 30.0;
            case BIKE -> 15.0;
            case TRUCK -> 50.0;
        };
        double nonPeakRate = switch (type) {
            case CAR -> 20.0;
            case BIKE -> 10.0;
            case TRUCK -> 30.0;
        };

        return peakHours * peakRate + nonPeakHours *
nonPeakRate;
    }
}
```

Worked example: Car, 7:30 AM → 1:15 PM

Duration
345 min → ⌈345/60⌉ = **6 hours** (always round up)

Time-based
Count from 7:00 (truncated): 7 (off-peak), 8, 9, 10, 11, 12 (peak)
= 5 × ₹30 + 1 × ₹20 = **₹170**

Event-based
6 h × ₹50 (car) = **₹300**

service/ParkingLot.java: unparkVehicle uses whichever strategy is set

```java
public void unparkVehicle(String ticketId, LocalDateTime exitTime) {
    Ticket ticket = activeTickets.get(ticketId);
    if (ticket == null) {
        System.out.println("  Invalid ticket ID.");
        return;
    }
    double fee = pricingStrategy.calculateFee(
            ticket.getVehicle().getType(), ticket.getEntryTime(), exitTime);

    ParkingSpot spot = floors.get(ticket.getFloorId()).getSpots().get(ticket.getSpotId());
    spot.vacate();
    activeTickets.remove(ticketId);
    System.out.println("  " + ticket.getVehicle() + " exited. Fee: Rs." + fee
            + " (" + pricingStrategy.getClass().getSimpleName() + ")");
}
```

▶ bash Phase-4-Pricing-Strategy/run.sh

```
═══ Phase 4: Pricing Strategy ═══
  CAR(KA01AB1234) parked at Floor1/F1S1. Ticket: T-33cee6b2
  CAR(KA01AB1234) exited. Fee: Rs.170.0 (TimeBasedPricing)
  CAR(KA01AB1234) parked at Floor1/F1S1. Ticket: T-702c4323
  CAR(KA01AB1234) exited. Fee: Rs.300.0 (EventBasedPricing)
```

Boundary detail worth mentioning in an interview

`isPeak` uses `!time.isAfter(PEAK_END)`, so an hour that *starts* at 17:00 is billed as peak. Whether 17:00 to 18:00 is peak is a business decision: ask, then encode it on purpose.

Checkpoint

Add a `WeekendPricing` flat ₹40/hour. Which files change? (New class + enum value + one factory case. `ParkingLot` is untouched: Open/Closed in action.)

PHASE 5

Payment Strategy

Strategy again · Dependency Inversion

Collect the money. Payment methods vary just like pricing did, so we reuse the same Strategy idea with a small PaymentProcessor.

`Phase-5-Payment-Strategy/`

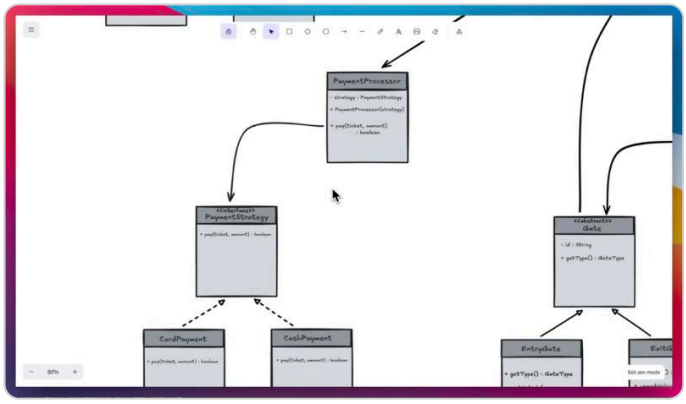
`+ PaymentStrategy «interface»` `+ CashPayment` `+ UpiPayment` `+ CardPayment` `+ PaymentProcessor`
`+ PaymentStrategyFactory` `+ PaymentMode / PaymentStatus (enums)` `~ Ticket (+paymentStatus)` `~ ExitGate (+mode)`
`~ ParkingLot.unparkVehicle` `~ Main`

Goal

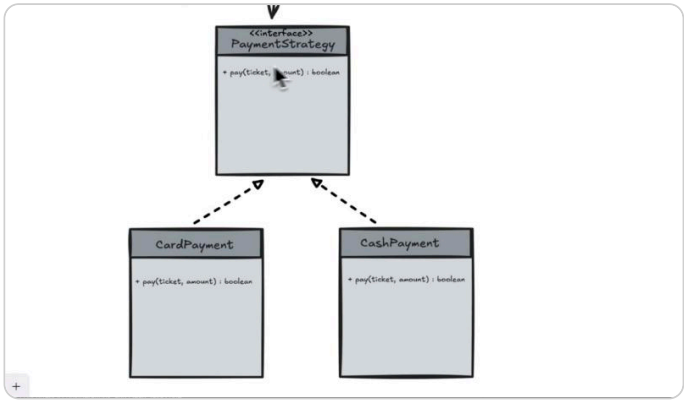
Requirement **E4**: UPI, card, cash and more later. A vehicle may leave **only after a successful payment**.

Concepts

- **Strategy (again):** `PaymentStrategy.processPayment(ticket, amount)` with one class per mode.
- **PaymentProcessor = context:** runs the strategy and records the result on the ticket (`PENDING → SUCCESS | FAILED`).
- **Factory maps user choice → strategy:** `PaymentStrategyFactory.get(PaymentMode.UPI)` .
- **Guard clause:** if payment fails, return early. The spot is not freed and the ticket stays active.

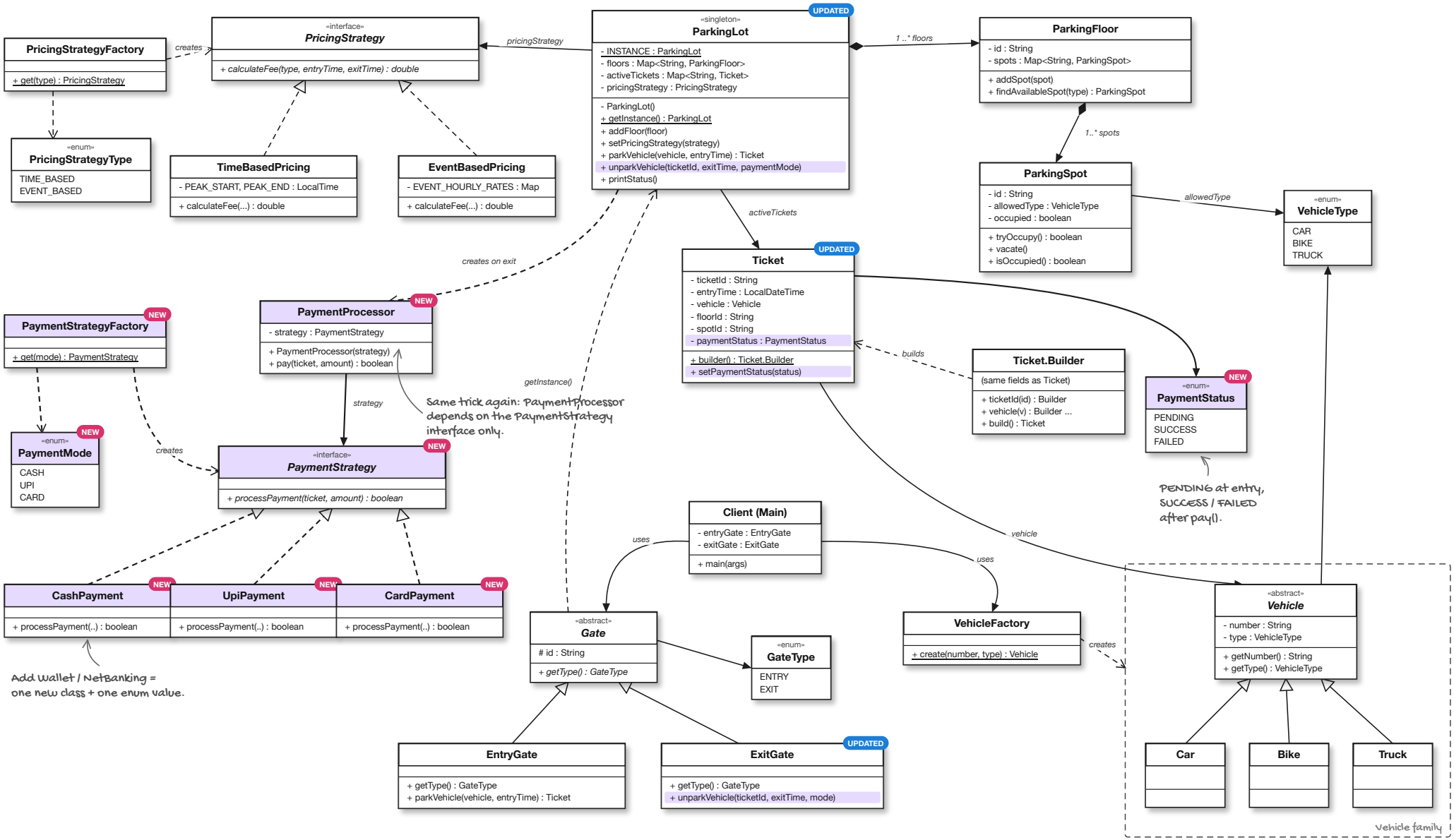


Video @ 21:00: PaymentProcessor holds a PaymentStrategy



Video @ 20:05: PaymentStrategy implementations

Phase 5 UML: payments complete the exit flow



strategy/payment/PaymentStrategy.java

```
public interface PaymentStrategy {
    boolean processPayment(Ticket
ticket, double amount);
}
```

strategy/payment/UpiPayment.java

```
public class UpiPayment implements
PaymentStrategy {
    @Override
    public boolean
processPayment(Ticket ticket, double
amount) {
        System.out.println("  Paid
Rs." + amount + " for ticket " +
ticket.getTicketId() + " via UPI.");
        return true;
    }
}
```

service/PaymentProcessor.java

```
public class PaymentProcessor {
    private final PaymentStrategy
strategy;

    public
PaymentProcessor(PaymentStrategy
strategy) {
        this.strategy = strategy;
    }

    public boolean pay(Ticket ticket,
double amount) {
        boolean success =
strategy.processPayment(ticket,
amount);

ticket.setPaymentStatus(success ?
PaymentStatus.SUCCESS :
PaymentStatus.FAILED);
        if (!success)
System.out.println("  Payment failed
for ticket: " + ticket.getTicketId());
        return success;
    }
}
```

service/ParkingLot.java: the full exit

```
public void unparkVehicle(String ticketId, LocalDateTime exitTime,
PaymentMode paymentMode) {
    Ticket ticket = activeTickets.get(ticketId);
    if (ticket == null) {
        System.out.println("  Invalid ticket ID.");
        return;
    }
    double fee = pricingStrategy.calculateFee(
            ticket.getVehicle().getType(), ticket.getEntryTime(),
exitTime);

    PaymentStrategy strategy =
PaymentStrategyFactory.get(paymentMode);
    PaymentProcessor processor = new PaymentProcessor(strategy);
    if (!processor.pay(ticket, fee)) {
        System.out.println("  Vehicle cannot exit. Payment
unsuccessful.");
        return;
    }

    ParkingSpot spot =
floors.get(ticket.getFloorId()).getSpots().get(ticket.getSpotId());
    spot.vacate();
    activeTickets.remove(ticketId);
    System.out.println("  " + ticket.getVehicle() + " exited. Fee
charged: Rs." + fee);
}
```

```
▶ bash Phase-5-Payment-Strategy/run.sh

=== Phase 5: Payment Strategy (full flow) ===
  CAR(KA01AB1234) parked at Floor1/F1S2. Ticket: T-69831173
  Payment status at entry: PENDING
-------------------------
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Occupied
    Spot F1S3 [TRUCK] - Free
    Spot F1S4 [CAR] - Free
-------------------------
  Paid Rs.170.0 for ticket T-69831173 via UPI.
```

```
  CAR(KA01AB1234) exited. Fee charged: Rs.170.0
  Payment status at exit: SUCCESS
-------------------------
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Free
    Spot F1S3 [TRUCK] - Free
    Spot F1S4 [CAR] - Free
```

A real bug in the original Lombok version

The original `Ticket` uses `@Builder` with `private PaymentStatus paymentStatus = PaymentStatus.PENDING;`. Lombok's builder **ignores field initialisers** unless the field has `@Builder.Default`, so new tickets actually start with `paymentStatus == null`. Our hand-written builder keeps `PENDING`, as the output above shows.

Checkpoint

1) Why is `PaymentProcessor` created per exit instead of being a field of `ParkingLot`? (The strategy depends on the user's choice at that moment.) 2) How would you add a retry for a FAILED payment?

PHASE 6

Concurrency

Race conditions · CAS · ConcurrentHashMap

Many gates, one spot, the same millisecond. Make spot booking atomic so two vehicles can never get the same spot.

`Phase-6-Concurrency/`

`+ RaceConditionDemo` `~ ParkingSpot: boolean → AtomicBoolean` `~ ParkingLot: activeTickets → ConcurrentHashMap` `~ Main (multi-threaded)`

Goal

Requirement **C2**: two vehicles must never get the same spot, even when entry gates run on different threads.

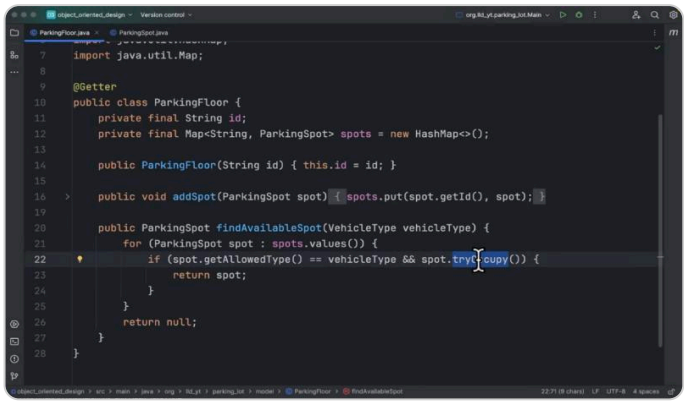
The race (check-then-act)

time	Thread A (bike 1)	Thread B (bike 2)	occupied
t1	`if (occupied)` → false		false
t2		`if (occupied)` → false	false
t3	`occupied = true` ✅		true
t4		`occupied = true` ✅	true

Both threads think they own spot F1S1 → double booking

The fix: compare-and-set

`occupied.compareAndSet(false, true)` does the check and the write as **one indivisible CPU operation**. Exactly one thread sees `true`; every other thread gets `false` and moves on to the next spot. No lock, no blocking.



Video @ 45:50: findAvailableSpot relies on tryOccupy()

model/ParkingSpot.java (final)

```java
public class ParkingSpot {
    private final String id;
    private final VehicleType allowedType;
    private final AtomicBoolean occupied = new
AtomicBoolean(false);

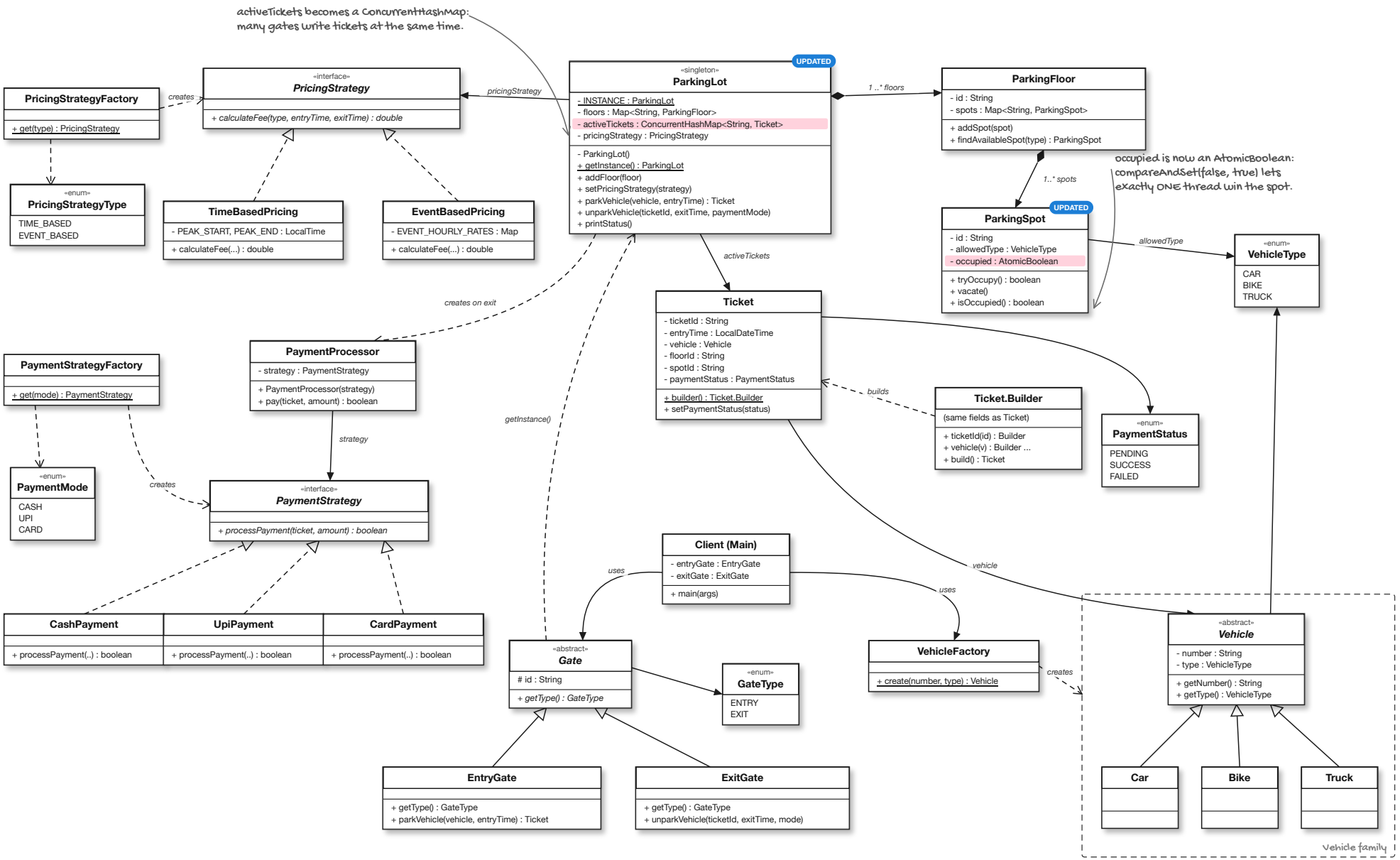
    public ParkingSpot(String id, VehicleType
allowedType) {
        this.id = id;
        this.allowedType = allowedType;
    }

    // CAS: only one thread can flip false → true,
so a spot is never double-booked.
    public boolean tryOccupy() {
        return occupied.compareAndSet(false, true);
    }

    public void vacate() { occupied.set(false); }

    public boolean isOccupied() { return
occupied.get(); }
    public String getId() { return id; }
    public VehicleType getAllowedType() { return
allowedType; }
}
```

Phase 6 UML: same classes, thread-safe internals



concurrency/RaceConditionDemo.java

```
static class NaiveSpot {
    private boolean occupied = false;

    boolean tryOccupy() {
        if (occupied) return false;
        Thread.yield(); // widen the gap between
"check" and "act" so the bug shows up
        occupied = true;
        return true;
    }
}

static class AtomicSpot {
    private final AtomicBoolean occupied = new
AtomicBoolean(false);

    boolean tryOccupy() { return
occupied.compareAndSet(false, true); }
}
```

Why not just `synchronized`?

`synchronized tryOccupy()` also works and is fine in an interview. CAS is lighter: no thread ever blocks, and the lock is per spot, so gates booking *different* spots never wait for each other. Note that the original code's `findAvailableSpot` already reserves inside the loop, so no lock is held across the scan.

Main.java: firing threads at the same instant

```
private static int race(List<Vehicle> vehicles,
ParkAction action) throws InterruptedException {
    CountDownLatch startGun = new CountDownLatch(1);
    AtomicInteger parked = new AtomicInteger();
    List<Thread> threads = new ArrayList<>();
    for (Vehicle v : vehicles) {
        Thread t = new Thread(() -> {
            try { startGun.await(); } catch
(InterruptedException e) { return; }
            if (action.park(v) != null)
parked.incrementAndGet();
        });
        threads.add(t);
        t.start();
    }
    startGun.countDown();
    for (Thread t : threads) t.join();
    return parked.get();
}
```

A `CountDownLatch` is the “starting gun”: all threads wait on `await()`, then `countDown()` releases them together to maximise contention.

```
▶ bash Phase-6-Concurrency/run.sh

═══ Phase 6: Concurrency ═══
-- Scenario 1: 2 bikes race for 1 bike spot --
  No spot available for vehicle type: BIKE
  BIKE(KA01AB1234) parked at Floor1/F1S1. Ticket: T-33fc1509
  Bikes parked: 1 (expected 1)
-- Scenario 2: 6 cars race for 2 car spots --
  No spot available for vehicle type: CAR
  No spot available for vehicle type: CAR
  No spot available for vehicle type: CAR
  CAR(CAR-0) parked at Floor1/F1S4. Ticket: T-6aba838d
  CAR(CAR-5) parked at Floor1/F1S2. Ticket: T-611b1601
  No spot available for vehicle type: CAR
  Cars parked: 2 (expected 2)
-------------------------
  Floor: Floor1
    Spot F1S1 [BIKE] - Occupied
    Spot F1S2 [CAR] - Occupied
    Spot F1S3 [TRUCK] - Free
    Spot F1S4 [CAR] - Occupied
-------------------------
  Paid Rs.180.0 for ticket T-33fc1509 via Card.
  BIKE(KA01AB1234) exited. Fee charged: Rs.180.0
  Floor: Floor1
    Spot F1S1 [BIKE] - Free
    Spot F1S2 [CAR] - Occupied
    Spot F1S3 [TRUCK] - Free
    Spot F1S4 [CAR] - Occupied

--- RaceConditionDemo ---
Rounds: 200, threads per round: 50
  NaiveSpot  double-booked in 112 rounds
  AtomicSpot double-booked in 0 rounds (always 0)
```

Other shared state

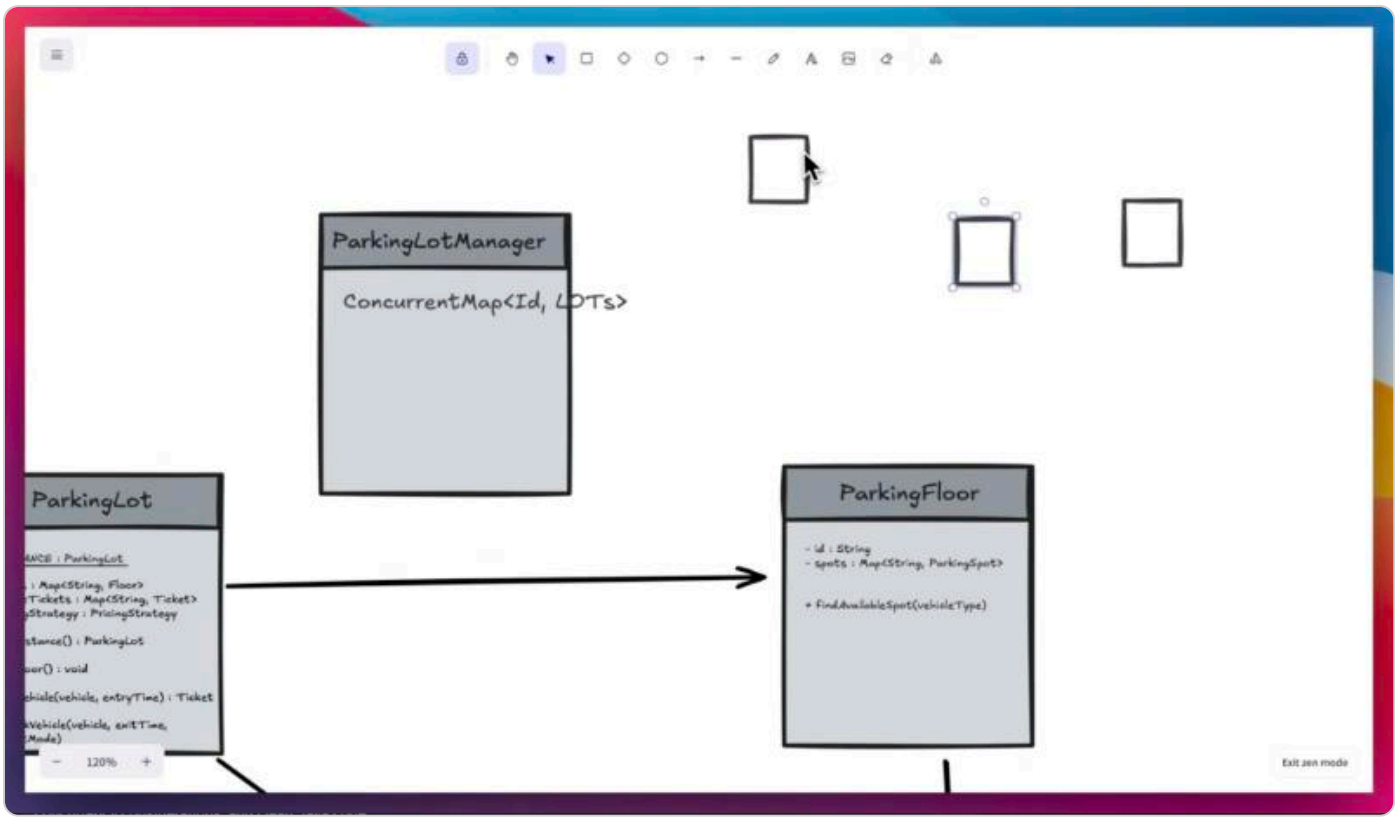
- `activeTickets` is written by every entry gate and read/removed by every exit gate → `ConcurrentHashMap`. (The original uses a plain `HashMap`, which can lose entries or corrupt itself under concurrent writes.)
- `floors` are only filled at start-up before any gate opens, so a normal map is fine.
- **Singleton creation** is already safe (eager static init).

Exercise: double exit

Two exit gates scan the same ticket at the same time: both `get()` it, both charge, both vacate. Fix: claim the ticket first with `Ticket t = activeTickets.remove(id); if (t == null) return;` (remove is atomic on a ConcurrentHashMap), then put it back if payment fails.

From the video: scaling to many lots

Requirement C1 says one lot *for now*. To support many, drop the Singleton and add a `ParkingLotManager` holding `ConcurrentMap<lotId, ParkingLot>`; gates get their lot from the manager. It is drawn dashed on the final diagram as a future extension.



Video @ 29:20: the ParkingLotManager idea next to ParkingLot and ParkingFloor

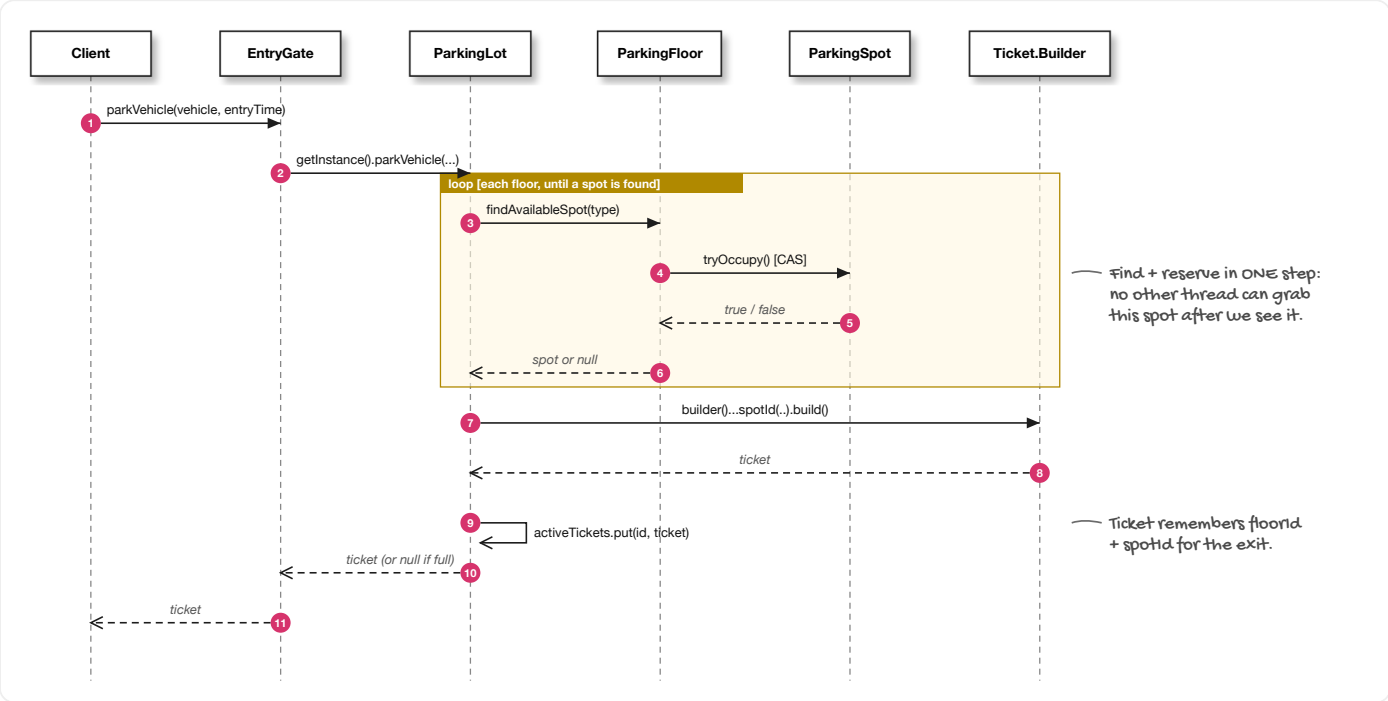
Checkpoint

1) Explain CAS in one sentence. 2) Why can't `volatile boolean` alone fix the race? (It makes writes visible to other threads, but the check and the write are still two separate steps.) 3) Where else could two threads collide? (Payment of the same ticket, the double exit above.)

Putting it together: the complete code flow

Two journeys run through the whole design: **entering** (get a ticket) and **leaving** (pay, then free the spot). The numbered steps below match the red circles in each diagram.

Flow 1: parkVehicle



1 `Main` builds a vehicle with `VehicleFactory.create("KA01AB1234", CAR)` and calls `entryGate.parkVehicle(car, entryTime)`.

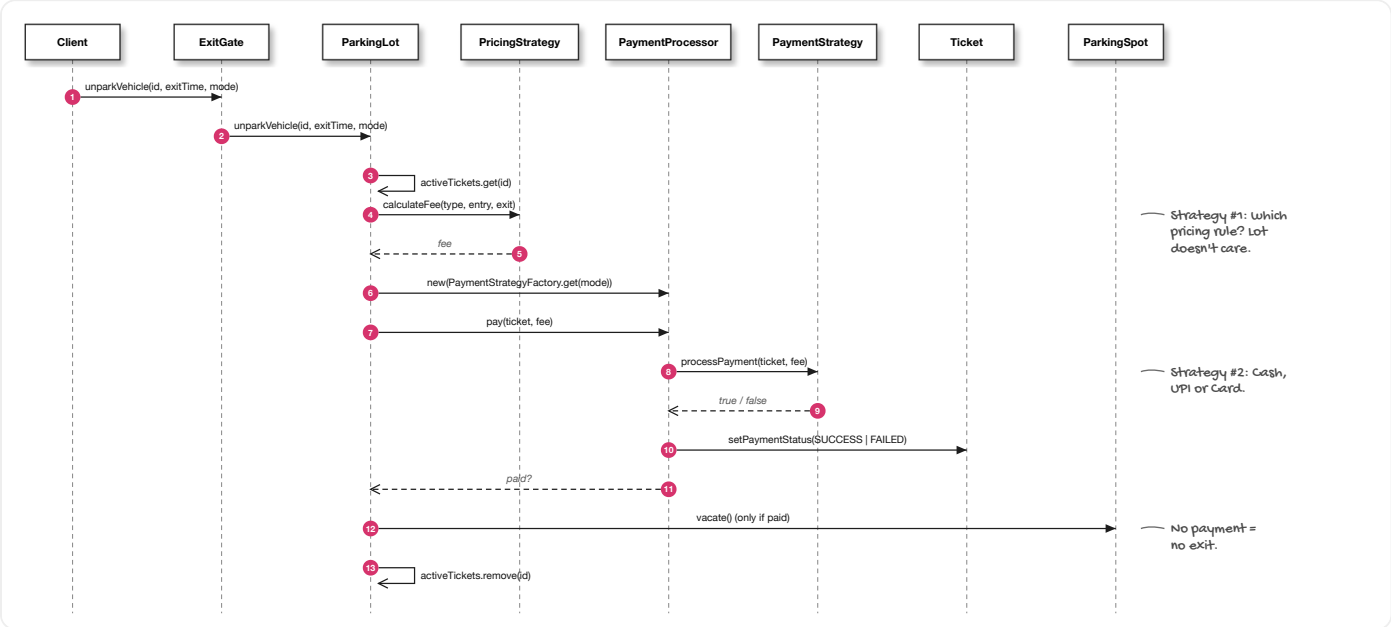
2 `EntryGate` delegates to `ParkingLot.getInstance().parkVehicle(...)`.

3–6 For each floor, `findAvailableSpot(type)` walks the spots. The first with the right `allowedType` *and* a successful `tryOccupy()` wins.

7–8 A `Ticket` is built with id, entry time, vehicle, floorId and spotId.

9–11 The ticket is stored in `activeTickets` and returned to the client (or `null` if every matching spot is taken).

Flow 2: unparkVehicle



1–3 The client hands the ticket id, exit time and payment mode to `ExitGate`, which forwards to the lot. An unknown id → "Invalid ticket ID."

4–5 **Pricing strategy**: `calculateFee(type, entry, exit)`, e.g. ₹170 time-based for 7:30 AM → 1:15 PM.

6–9 **Payment strategy**: the factory turns `UPI` into `UpiPayment`; `PaymentProcessor.pay()` runs it.

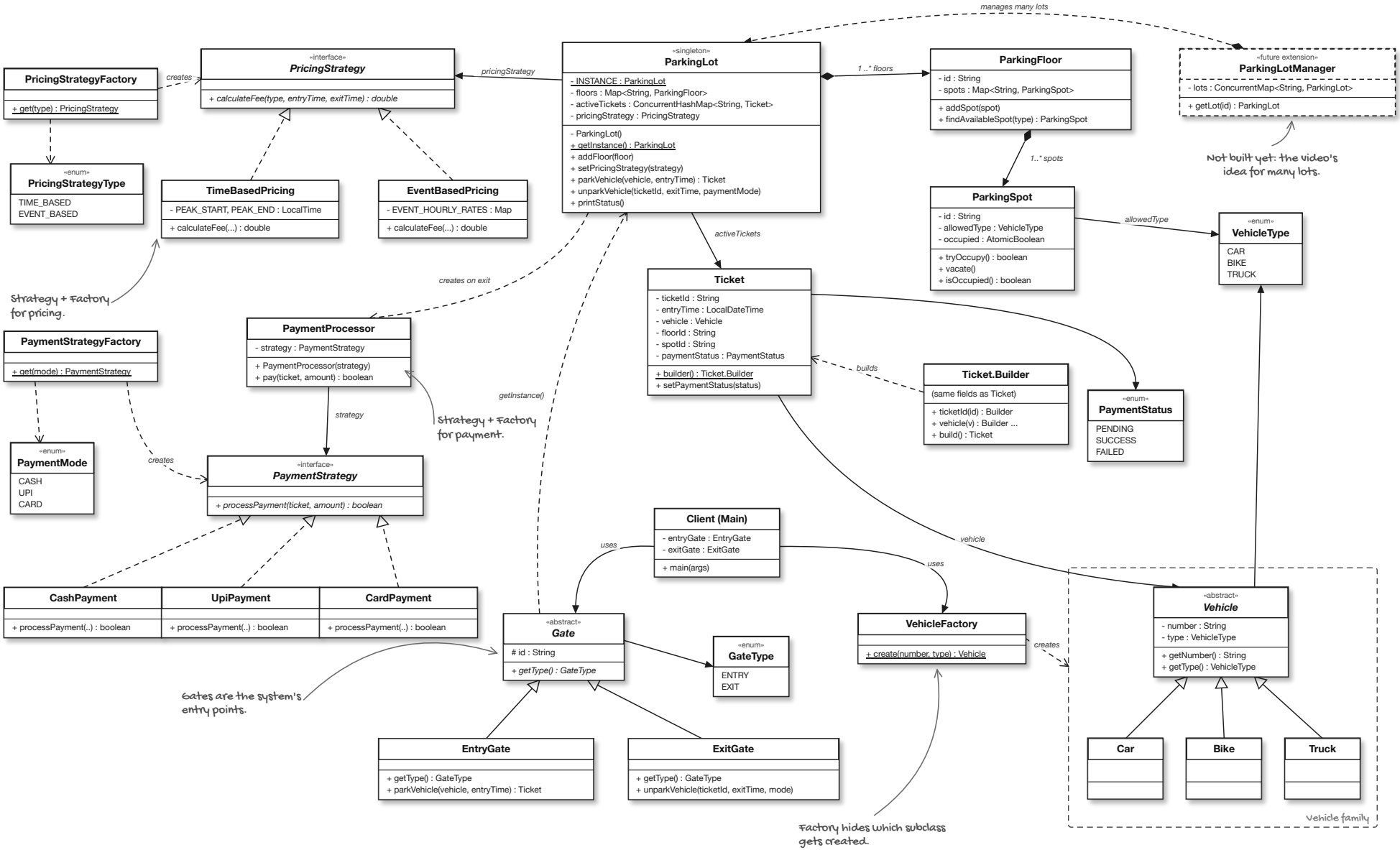
10–11 The processor stamps `SUCCESS` / `FAILED` on the ticket. On failure the method returns, so the car stays inside.

12–13 On success: `spot.vacate()` (found directly through floorId/spotId) and the ticket is removed from `activeTickets`.

Explain it in 30 seconds

"Gates are entry points. The Singleton ParkingLot owns floors, which own spots. Parking finds and atomically reserves a matching spot, then issues a Ticket built with a Builder. Exiting asks a pluggable PricingStrategy for the fee, a pluggable PaymentStrategy collects it, and only then is the spot freed. Factories create vehicles and strategies from enums."

The complete design (all six phases stitched together)



Notes on the reference code

The phase folders follow the original repository class for class. Changes were made only to drop the Lombok dependency, make the demos predictable, and fix the issues below.

Original	In these notes	Why
Lombok `@Getter`, `@Builder`, `@Data`	Plain Java getters + hand-written Builder	Compiles with only a JDK, and you see what Lombok generates.
`@Builder` + `paymentStatus = PENDING`	Builder keeps `PENDING`	Lombok drops the initializer without `@Builder.Default` → `null`.
`activeTickets` is a `HashMap`	`ConcurrentHashMap` (Phase 6)	Written by many gate threads at once.
`HashMap` for floors/spots	`LinkedHashMap`	Predictable "first free spot" and demo output.
UUID ticket ids	`"T-" + first 8 chars of a UUID`	Readable output; same idea.
`Main` with commented-out flows (one uses a non-existent `VehicleType.SEDAN`)	One runnable `Main` per phase	Each phase demonstrates exactly what it adds.

Where to take it next

Many lots
Replace the Singleton with `ParkingLotManager` + constructor injection.

Spot size compatibility
Let a bike use a car spot when bike spots are full: a `SpotAssignmentStrategy`.

Display boards
Observer pattern: spots publish free/occupied events, boards subscribe.

Faster search
Per-type queues of free spots instead of scanning every spot.

Lost ticket / penalties
Decorator around a `PricingStrategy` (`LostTicketPenalty`).

Persistence
Repository interfaces for tickets and spots; the lot depends on the interface.

Self-test: interview questions

Q. Why is ParkingLot a Singleton, and what would you change for multiple lots?
One physical lot means one shared state for all gates. For many lots, remove the Singleton and inject lots from a `ParkingLotManager` (a map of id → lot).

Q. Where is the Strategy pattern used, and why not if/else?
Pricing and payment. An if/else chain would put every rule in `ParkingLot` and force edits for each new rule (breaks Open/Closed); strategies are added as new classes and can be swapped at runtime.

Q. Factory vs. Strategy: what's the difference here?
The Factory *chooses and creates* an object from an enum; the Strategy *is* the interchangeable behaviour. They work together: `PaymentStrategyFactory.get(UPI)` returns the `UpiPayment` strategy.

Q. How do you guarantee no two cars get the same spot?
`tryOccupy()` uses `AtomicBoolean.compareAndSet(false, true)`; only one thread can win. Finding and reserving happen in the same step inside `findAvailableSpot`.

Q. Why does Ticket store floorId and spotId?
So exit is O(1): `floors.get(floorId).getSpots().get(spotId).vacate()`, with no search.

Q. Why is Vehicle abstract and what does VehicleType add?
No one parks a generic vehicle. The enum gives a cheap, switchable key used by spots, pricing and factories.

Q. What happens if payment fails?
Status becomes FAILED, the method returns early, the spot stays occupied and the ticket stays active, so the driver can retry.

Q. Builder: what problem does it solve here?
Readable construction of a many-field, mostly immutable Ticket, with validation in one place (`build()`).

Q. Name a remaining concurrency risk.
A double exit on the same ticket. Claim it atomically with `activeTickets.remove(id)` before charging.

Made for learning: re-draw every diagram yourself once, then build the final design from a blank file. 🚗